

LivingDocs System

Automated Documentation Generation & Synchronization
Platform

Capstone Project Report

Software Engineering Project Team

Department of Computer Science & Software Engineering

September 7, 2026

Contents

0.1	Introduction	1
0.1.1	Project Overview	1
0.1.2	Objectives	1
0.1.3	Project Scope	1
1	Software Requirements Specification	3
1.1	Introduction	3
1.1.1	Document Purpose	3
1.2	System Scope	3
1.3	System Objectives	4
1.4	Target User Personas	4
1.5	User Requirements	5
1.5.1	Developer (Author)	5
1.5.2	Staff (Reviewer)	6
1.5.3	Manager (Approver)	6
1.5.4	Technical Lead / Documentation Owner	7
1.5.5	Administrator	7
1.6	Functional Requirements	7
1.6.1	Account and Authentication	7
1.6.2	GitHub Integration	8
1.6.3	AI Source-Code Reading & Documentation Generation	8
1.6.4	Automatic Documentation Update	9
1.6.5	Documentation Drift Detection and Review	9
1.6.6	Change Logs and Version History	10
1.6.7	Knowledge Base	10
1.6.8	Review and Approval Workflows	10
1.6.9	Governance and Administration	11
1.7	Non-Functional Requirements	11
1.7.1	Performance	11
1.7.2	Availability & Scalability	12
1.7.3	Security & Reliability	12

1.8	Business Rules	12
1.9	Summary	13
2	System Workflow & Automation Pipeline	14
2.1	Overview & Operational Objectives	14
2.2	Event-Driven Trigger Mechanism	15
2.2.1	Trigger Sources	15
2.2.2	Event Ingestion & Message Queuing	15
2.3	Core Execution Pipeline	15
2.3.1	Stage 1: Code Diff & AST Analysis	15
2.3.2	Stage 2: Semantic RAG Context Augmentation	16
2.3.3	Stage 3: AI Generation & Post-Processing	16
2.4	Human-in-the-Loop Approval Workflow	16
2.4.1	Proposal Creation & Side-by-Side Review Workspace	17
2.5	State Tracking, Resilience & Auditing	17
2.5.1	Workflow Finite State Machine (FSM)	17
2.5.2	Resilience, Retries, and Audit Logs	17
2.6	Summary	18
2.7	GitHub Connection & Repository Management	18
2.7.1	Introduction	18
2.7.2	GitHub Connection Using OAuth2	18
2.7.3	Repository Management	19
2.7.4	CI/CD Integration with GitHub Actions	20
2.7.5	Pull Request and Commit Tracking	21
2.7.6	Security and Error Handling	21
2.8	AI-Driven Documentation Generation Engine	22
2.8.1	Introduction	22
2.8.2	AI Architecture and Model Integration	23
2.8.3	Automated Documentation Generation Process	24
2.8.4	Retrieval-Augmented Generation (RAG) System	25
2.8.5	Prompt Engineering and Quality Control	25
2.8.6	Security, Privacy and Error Handling	26
3	Documentation Drift Detection	27
3.1	Overview	27
3.2	Objectives & Scope	27
3.3	Functional Details	27
3.3.1	GitHub Event Ingestion (FR01)	27
3.3.2	AST Source Code Parsing (FR02)	28
3.3.3	Code-Doc Comparison & Scoring (FR03, FR04)	28

3.3.4	Report Generation & Alerting (FR05, FR06)	28
3.4	Processing Workflow	29
3.5	Acceptance Criteria	29
4	Automatic Documentation Update on Source-Code Change	30
4.1	Module Overview & Objectives	30
4.2	Functional Requirements	31
4.3	Detailed Use Cases	31
4.3.1	UC-AU-01: Automatic Update Triggering on Source Code Change	31
4.3.2	UC-AU-02: Diff Comparison & Developer Acceptance	32
4.4	System Architecture & Processing Flow	32
4.5	Database Schema	32
5	Knowledge Base & Vector Retrieval Management	34
5.1	Module Overview & Objectives	34
5.2	Functional Requirements Matrix	35
5.3	Knowledge Processing & Retrieval Pipeline	35
5.3.1	Document Ingestion Flow	35
5.3.2	RAG Retrieval Sequence	35
5.4	Database Schema	36
6	System Administration (Administrator)	37
6.1	User Management & Role-Based Access Control (RBAC)	37
6.2	Platform & Version Control System Integration	37
6.3	AI Engine Service Configuration	38
6.4	Auto-Update Policies, Audit Logs & System Operations	38
7	System Testing & Quality Assurance Plan	40
7.1	Introduction	40
7.2	Testing Objectives & Scope	40
7.2.1	Testing Objectives	40
7.2.2	Testing Scope	41
7.3	Testing Strategy & Levels	41
7.4	Testing Environment & Tools	42
7.5	Defect Management Workflow	42
7.6	Detailed Test Cases	42
7.7	Summary	45
7.8	Conclusion	45
7.8.1	Project Summary	45
7.8.2	Project Achievements	45

7.8.3	Project Limitations	46
7.8.4	Future Development	46
7.8.5	Conclusion	46

0.1 Introduction

0.1.1 Project Overview

In modern software development, technical documentation plays a crucial role in helping development, testing, and maintenance teams work effectively. However, in many real-world projects, documentation is often not updated promptly when source code changes, leading to inconsistencies between documentation and the deployed system.

To address this issue, the team proposes building the **LivingDocs** system – an intelligent platform for software documentation management, capable of directly connecting to GitHub to track source code changes and automatically suggest documentation updates using Artificial Intelligence (AI).

LivingDocs helps reduce manual workload in the documentation process while improving accuracy, traceability, and consistency between source code and technical documentation.

0.1.2 Objectives

The objectives of the system include:

- Automatically detect source code changes on GitHub.
- Generate technical documentation using AI.
- Detect inconsistencies between source code and documentation.
- Support collaboration among multiple team members in the same project.
- Manage documentation versions and change history.

0.1.3 Project Scope

The LivingDocs system focuses on supporting software development teams in building and maintaining technical documentation. The main functions include:

- GitHub Integration.
- AI Documentation Generation.
- Automatic Documentation Update.
- Documentation Drift Detection.
- Knowledge Base Management.
- Workflow Collaboration.

- System Administration.
- Testing and Deployment.

Chapter 1

Software Requirements Specification

1.1 Introduction

1.1.1 Document Purpose

This document specifies the software requirements for the **LivingDocs – Design and Implementation of an AI-Assisted Self-Updating Document System for Engineering Teams** platform.

LivingDocs is an AI-powered documentation maintenance platform engineered to detect documentation drift, automatically generate and update technical documentation from source code, and enforce a multi-tiered Human-in-the-Loop review workflow.

This specification focuses on:

- User requirements and target personas.
- Functional Requirements (FR).
- Non-Functional Requirements (NFR).
- User roles and access control mechanisms.
- Documentation drift detection and management workflows.
- Verification, auditing, and document versioning controls.

Note: This document excludes low-level architectural blueprints, database schema design, or specific code implementation details, which are addressed in subsequent system design modules.

1.2 System Scope

LivingDocs automates the continuous maintenance of technical documentation across software engineering projects.

Key capabilities include:

- Seamless integration with GitHub repositories via OAuth and Webhooks.
- Continuous ingestion of commit updates and Pull Requests.
- Automated static code and AST analysis.
- Bidirectional mapping between code entities and documentation artifacts.
- Multi-classification of documentation drift (Referential, Signature, Semantic).
- AI-assisted document generation and context-aware auto-updates.
- Granular version history and complete audit traceability.
- Multi-tiered review workflows involving Staff Reviewers and Manager Approvers.
- Searchable Knowledge Base linking documentation directly to live code.
- System-wide governance and administrative controls.

1.3 System Objectives

The primary objectives of LivingDocs encompass:

- Minimizing technical documentation debt in fast-paced development environments.
- Proactively identifying outdated or contradictory documentation.
- Detecting structural reference breaks as well as semantic behavioral shifts.
- Reducing manual overhead required for writing and updating technical docs.
- Ensuring high precision and global consistency across project documentation.
- Maintaining strict human governance over AI-generated outputs.
- Guaranteeing complete auditability and version rollback capabilities.

1.4 Target User Personas

The system defines five primary operational roles:

Role	Description
Developer (Author)	Engineers who connect repositories, initiate AI documentation generation, review drift alerts, and submit documentation drafts for review.
Staff (Reviewer)	Technical personnel responsible for conducting the initial technical review of AI-generated or auto-updated documentation.
Manager (Approver)	Decision-makers responsible for final approval, publication, or direct repository commit authorization.
Technical Lead / Owner	Administrators managing documentation standards, drift policies, custom templates, and quality compliance.
Administrator	System administrators managing user accounts, RBAC policies, GitHub integration setups, and backend infrastructure.

1.5 User Requirements

1.5.1 Developer (Author)

Developers require the capability to:

- Authenticate via credentials or GitHub OAuth.
- Manage personal profiles and notification preferences.
- Link GitHub accounts and select repositories for workspace ingestion.
- Monitor repository sync status and trigger manual re-synchronization.
- View ingested commits and Pull Requests.
- Generate technical documentation from source code via AI.
- Select customized documentation templates and preview draft outputs.
- Identify code entities lacking documentation coverage.
- Edit AI-generated drafts and inspect grounding evidence before submission.
- Submit drafts into review workflows and monitor progress.

- Inspect documentation drift alerts, severity levels, and associated diffs.
- Compare document versions and perform version rollbacks.
- Search the knowledge base for linked code entities and documentation.

1.5.2 Staff (Reviewer)

Staff members require the capability to:

- Access a centralized review queue.
- Inspect documentation side-by-side with code grounding evidence.
- Compare proposed versions against the latest approved baseline.
- Verify compliance with designated documentation templates and code truth.
- Inline review comments, request modifications, or make minor corrections.
- Approve drafts for Manager escalation or reject drafts with actionable feedback.
- Filter, sort, and track historical review logs.

1.5.3 Manager (Approver)

Managers require the capability to:

- Access high-priority approval queues containing Staff-reviewed docs.
- Review Staff feedback, change diffs, and complete audit trails.
- Grant final approval for automatic repository commits or public publishing.
- Reject submissions or mandate further revisions.
- Execute emergency rollbacks to prior approved versions.
- Configure approval policies based on drift severity (e.g., Critical drift).
- Delegate or reassign approval authority.

1.5.4 Technical Lead / Documentation Owner

Technical Leads require the capability to:

- Organize and manage documentation collections and ownership.
 - Define drift detection sensitivity thresholds and update policies.
 - Monitor the Documentation Health Dashboard (coverage, freshness, drift metrics).
 - View code-to-documentation dependency graphs.
 - Export health compliance and audit reports.
 - Create, clone, modify, and publish version-controlled documentation templates.
- Configure mandatory template headings, sections, and placeholders.

1.5.5 Administrator

Administrators require the capability to:

- Manage user accounts, roles, and granular RBAC permissions.
 - Configure workspace configurations, GitHub Apps, OAuth credentials, and webhooks.
 - Define supported programming languages and parser configurations.
 - Set AI inference endpoints, token quotas, and vector indexing parameters.
 - Monitor event processing pipelines, background jobs, and system health.
- Inspect system-wide administrative audit logs.

1.6 Functional Requirements

1.6.1 Account and Authentication

ID	Requirement Description
FR01	The system shall allow users to register new accounts.
FR02	The system shall allow users to securely log in and log out.
FR03	The system shall support authentication via Email/Password and GitHub OAuth.
FR04	The system shall enable users to manage profiles and notification preferences.

1.6.2 GitHub Integration

ID	Requirement Description
FR05	The system shall allow developers to link GitHub accounts via OAuth2.
FR06	The system shall display requested access scopes and permissions.
FR07	The system shall allow users to select and import target repositories.
FR08	The system shall support disconnecting repositories and revoking access tokens.
FR09	The system shall display synchronization status, default branch, and last sync timestamp.
FR10	The system shall support manual trigger of repository re-synchronization.
FR11	The system shall track ingested commits and Pull Request events.
FR12	The system shall alert administrators upon Webhook or token expiration.

1.6.3 AI Source-Code Reading & Documentation Generation

ID	Requirement Description
FR13	The system shall support doc generation at file, module, or repository levels.
FR14	The system shall analyze AST and code structure to synthesize documentation.
FR15	The system shall allow selecting predefined documentation templates.
FR16	The system shall identify code entities lacking documentation coverage.
FR17	The system shall provide a real-time preview of AI-generated drafts.
FR18	The system shall allow Developers to edit drafts prior to submission.
FR19	The system shall display grounding evidence (code references) for generated docs.

ID	Requirement Description
FR20	The system shall allow submitting generated drafts into the review workflow.
FR21	The system shall display the current workflow state of the document.

1.6.4 Automatic Documentation Update

ID	Requirement Description
FR22	The system shall detect affected documentation upon Pull Request or commit updates.
FR23	The system shall automatically generate updated document drafts and increment versions.
FR24	The system shall allow toggling auto-update features per repository or module.
FR25	The system shall provide side-by-side diff views between auto-updated drafts and published docs.
FR26	The system shall highlight specific triggering commits and diff chunks.
FR27	The system shall support accepting, editing, or dismissing auto-updated suggestions.
FR28	The system shall allow manual re-generation of documentation updates on demand.
FR29	The system shall support one-click rollbacks to pre-update states.

1.6.5 Documentation Drift Detection and Review

ID	Requirement Description
FR30	The system shall analyze Pull Request diffs to detect documentation drift.
FR31	The system shall present grounding evidence for every drift alert.
FR32	The system shall classify drift into Referential, Signature, or Semantic categories.

ID	Requirement Description
FR33	The system shall assign drift severity levels: Low, Medium, High, or Critical.
FR34	The system shall generate context-aware fix suggestions for detected drift.
FR35	The system shall support manual trigger of drift analysis for specific files or PRs.
FR36	The system shall maintain historical logs of drift occurrences and resolution actions.

1.6.6 Change Logs and Version History

ID	Requirement Description
FR37	The system shall record full change histories for every document.
FR38	The system shall allow comparing differences across any two arbitrary versions.
FR39	The system shall log acting user/role, timestamp, and trigger event for each version.
FR40	The system shall allow restoring or rolling back documents to previous versions.

1.6.7 Knowledge Base

ID	Requirement Description
FR41	The system shall provide full-text search across documentation and linked code entities.
FR42	The system shall render interactive dependency graphs between code and documentation.
FR43	The system shall allow bidirectional navigation between code files and docs.

1.6.8 Review and Approval Workflows

ID	Requirement Description
FR44	The system shall provide a dedicated Staff Review Queue.
FR45	Staff shall be able to inspect grounding evidence, change logs, and template rules.
FR46	Staff shall be able to approve, reject, inline-edit, or request revisions.
FR47	The system shall provide a Manager Approval Queue for Staff-approved documents.
FR48	Managers shall have final authority to publish or execute automated repository commits.
FR49	The system shall enforce mandatory approval rules before committing changes.
FR50	The system shall record complete, immutable audit logs of all approval decisions.

1.6.9 Governance and Administration

ID	Requirement Description
FR51	Tech Leads shall be able to configure drift detection thresholds and quality policies.
FR52	The system shall display real-time documentation health dashboards and freshness metrics.
FR53	Tech Leads shall be able to manage, version, and preview document templates.
FR54	Administrators shall manage RBAC permissions, language parsers, and AI settings.
FR55	Administrators shall monitor background processing pipelines, job statuses, and system health.

1.7 Non-Functional Requirements

1.7.1 Performance

ID	Requirement Description
NFR01	The system shall complete drift analysis for a PR within 2 minutes for repos up to 100k LOC.

ID	Requirement Description
NFR02	The system shall process at least 20 concurrent PR events without performance degradation.
NFR03	Standard API response times shall remain below 2 seconds under normal workload.
NFR04	Full-text search queries within the Knowledge Base shall return results within 1 second.

1.7.2 Availability & Scalability

ID	Requirement Description
NFR05	The system shall maintain an operational uptime availability of at least 99.5%.
NFR06	Background worker failures shall not impact user-facing web services.
NFR07	AI processing workers shall support horizontal auto-scaling based on queue depth.

1.7.3 Security & Reliability

ID	Requirement Description
NFR08	All API endpoints and communications shall enforce HTTPS/TLS encryption.
NFR09	Access shall be strictly controlled via Role-Based Access Control (RBAC).
NFR10	Sensitive credentials and integration keys shall be encrypted at rest.
NFR11	System audit logs shall be immutable and retention-configurable.
NFR12	Drift detection algorithms shall yield deterministic evaluations for identical code diffs.

1.8 Business Rules

ID	Business Rule
BR01	All AI-generated or auto-updated documentation must undergo multi-tiered human review before publication.
BR02	Staff members serve as the first-line technical reviewers.
BR03	Manager approval is mandatory for documents marked with high sensitivity or critical drift impact.
BR04	Unapproved documentation drafts shall never be automatically committed to primary repository branches.
BR05	Critical drift alerts may trigger configurable Pull Request merge blocking policies.

1.9 Summary

This Software Requirements Specification defines the functional capabilities, non-functional constraints, user roles, and operational business rules governing the LivingDocs platform. It serves as the authoritative baseline for architecture design, system implementation, and testing phases.

Chapter 2

System Workflow & Automation Pipeline

2.1 Overview & Operational Objectives

The **Workflow & Automation Pipeline** acts as the operational backbone of the LivingDocs system. It seamlessly orchestrates interactions between GitHub code tracking, Abstract Syntax Tree (AST) parsing, Retrieval-Augmented Generation (RAG) querying, and the AI Generation Engine. A robust automation workflow ensures that technical documentation is continuously synchronized without interrupting the daily productivity of software engineering teams.

Within the LivingDocs ecosystem, the workflow pipeline is engineered to achieve the following operational goals:

- **End-to-End Lifecycle Automation:** Automate the documentation update process from initial commit detection to final published documentation release.
- **Event-Driven Responsiveness:** Guarantee rapid response times utilizing a real-time event-driven architecture.
- **Human-in-the-Loop Governance:** Integrate flexible human review checkpoints (AI → Staff Reviewer → Manager Approver) to ensure accuracy, safety, and content quality.
- **Resilience & Fault Tolerance:** Maintain high system fault tolerance with automated state recovery mechanisms during network failures or third-party service outages.
- **Comprehensive Auditability:** Provide transparent audit logs and end-to-end traceability for every processing stage within the execution pipeline.

2.2 Event-Driven Trigger Mechanism

LivingDocs utilizes an Event-Driven Architecture (EDA) to instantly trigger processing pipelines upon detecting source code changes across development environments.

2.2.1 Trigger Sources

The execution pipeline can be initiated through multiple channels:

- **GitHub Webhooks:** Automatically triggered upon receiving `push`, `pull_request`, or `release` webhook payloads from monitored repositories.
- **Manual On-Demand Triggers:** Triggered directly by developers or administrators requesting documentation regeneration for specific code entities via the web interface.
- **Scheduled Background Jobs:** Periodic cron tasks that scan repositories to detect undocumented drift across large codebases.
- **CI/CD Integration:** Triggered automatically via GitHub Actions pipelines following successful build and test execution stages.

2.2.2 Event Ingestion & Message Queuing

When an incoming event reaches the LivingDocs system:

1. The `Webhook Receiver` service verifies HMAC security credentials via the `X-Hub-Signature-256` header to authenticate the payload.
2. Validated events are parsed, classified, and published to message broker topics (e.g., Apache Kafka / RabbitMQ).
3. Message queuing decouples ingestion from heavy background processing (AST parsing and LLM inference), preventing system bottlenecks during concurrent commit pushes.

2.3 Core Execution Pipeline

The core execution pipeline transforms raw source code diffs into structured documentation artifacts through three sequential stages:

2.3.1 Stage 1: Code Diff & AST Analysis

- Extract file modification lists (`added`, `modified`, `deleted`) from commit or Pull Request payloads.
- Execute AST parsing to pinpoint affected structural entities (Classes, Method Signatures, API Endpoints, Parameters).
- Filter out non-code assets (e.g., temporary configurations, static media, raw binaries).

2.3.2 Stage 2: Semantic RAG Context Augmentation

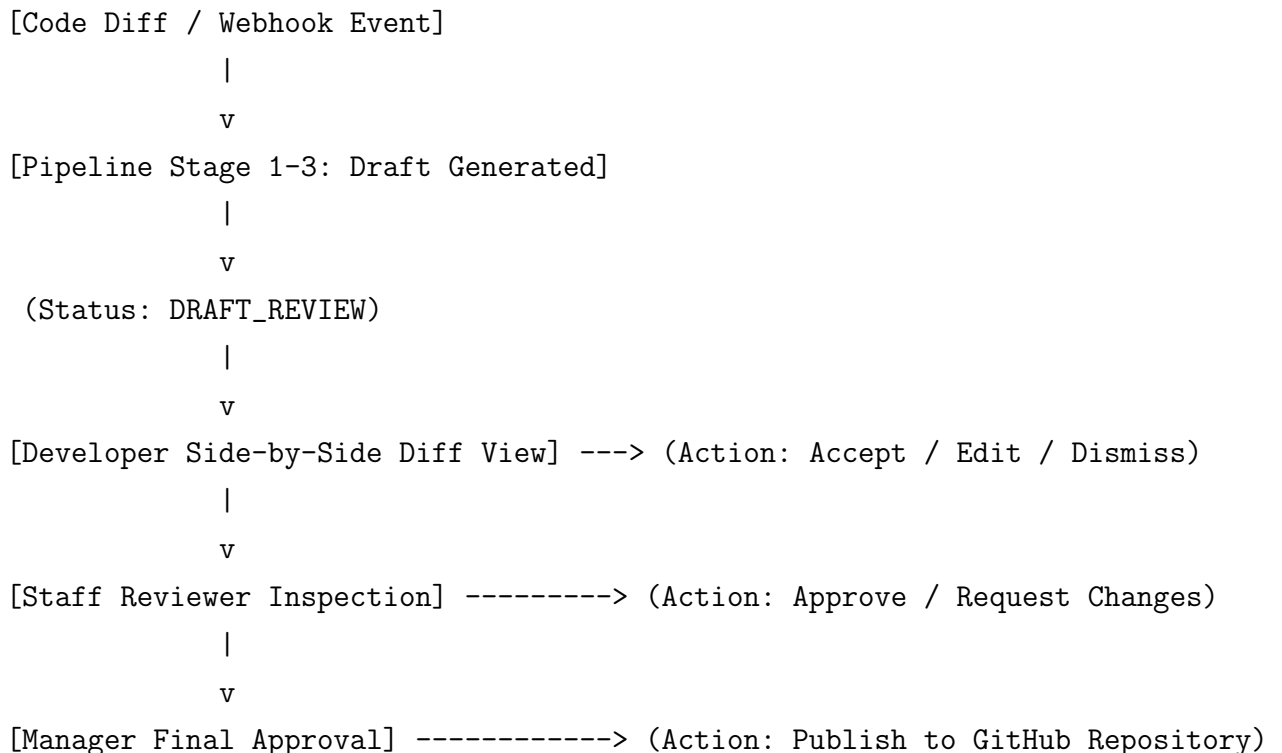
1. Extract old and new code snippets for modified entities.
2. Query the Vector Database using hybrid semantic search to retrieve linked documentation chunks and legacy contextual knowledge.
3. Assemble an enriched prompt context payload containing: Source Code Diff + Retained Vector Chunks + Standardized Markdown Formatting Templates.

2.3.3 Stage 3: AI Generation & Post-Processing

- The Python AI Engine processes the assembled context package to generate updated documentation drafts.
- The Syntax Validator verifies Markdown, LaTeX formula correctness, and Mermaid diagram formatting.
- The Link Checker verifies internal relative paths and cross-references to eliminate broken links.

2.4 Human-in-the-Loop Approval Workflow

To maintain high editorial quality, LivingDocs implements a Human-in-the-Loop approval workflow prior to committing updates back to the primary codebase.



2.4.1 Proposal Creation & Side-by-Side Review Workspace

Upon successful generation, LivingDocs creates a versioned proposal instead of immediately overwriting active files:

- For Pull Request triggers, the system posts inline PR comments or opens a dedicated documentation update PR.
- For web application interactions, drafts are assigned the `PENDING_DEV_ACCEPTANCE` or `IN_REVIEW` status.
- Reviewers use an interactive Side-by-Side Diff View highlighting line-by-line additions (green), deletions (red), and modifications (yellow).

2.5 State Tracking, Resilience & Auditing

2.5.1 Workflow Finite State Machine (FSM)

Every pipeline execution instance is managed as a Finite State Machine featuring six defined operational states:

- **QUEUED:** The event payload is queued in the message broker awaiting processing.
- **ANALYZING:** Code parsing, AST extraction, and file filtering are actively executing.
- **GENERATING:** The AI Engine is querying the Vector Store and generating candidate documentation text.
- **AWAITING_REVIEW:** Documentation draft generation is complete and waiting for developer/staff evaluation.
- **COMPLETED:** The proposal has been fully approved and merged into the main repository.
- **FAILED:** An unhandled execution error occurred during pipeline processing.

2.5.2 Resilience, Retries, and Audit Logs

- **Automated Retries:** Transient infrastructure failures (e.g., API rate limits, temporary network timeouts) trigger exponential backoff retries (up to 3 attempts).
- **Dead Letter Queue (DLQ):** Events that fail after maximum retry attempts are routed to a DLQ for administrator manual inspection.
- **Audit Trails:** Every state transition, execution timestamp, and user action is logged to PostgreSQL `document_change_logs` tables to support compliance auditing.

2.6 Summary

The system workflow and automation pipeline forms the core backbone of LivingDocs. By combining event-driven triggers, AST/RAG execution stages, multi-tier Human-in-the-Loop review gates, and finite state resilience, LivingDocs delivers continuous, trustworthy, and fault-tolerant technical documentation updates.

2.7 GitHub Connection & Repository Management

2.7.1 Introduction

GitHub is used as the source code management platform for the LivingDocs project. All documentation, source code, and change history are centrally stored on GitHub to support team collaboration more effectively.

In the Software Engineering project, integrating GitHub with the **LivingDocs** system is necessary to achieve the following goals:

- Centralized and consistent source code management among team members.
- Synchronize source code changes with the automated documentation system.
- Track commit history and Pull Requests to control development progress.
- Support review processes and software quality checks before deployment.
- Reduce outdated documentation compared to actual source code.

Thanks to direct integration with GitHub, LivingDocs can automatically detect source code changes and update corresponding documentation, maintaining synchronization between technical documents and the evolving system.

2.7.2 GitHub Connection Using OAuth2

To allow users to connect their GitHub accounts to LivingDocs without providing passwords directly, the system uses the OAuth2 Authorization Code Flow supported by GitHub.

OAuth2 Mechanism OAuth2 allows users to grant LivingDocs access without sharing their GitHub password. The system only receives an **access token** with the scope approved by the user.

Login and Authentication Process

1. The user selects the **Connect GitHub** function.
2. The system redirects to GitHub's authentication page.
3. The user logs in to GitHub and grants repository access.

4. GitHub returns an **authorization code**.
5. LivingDocs sends this code to GitHub to obtain an **access token**.
6. The token is encrypted and securely stored in the system.

Benefits of OAuth2

- **High security:** GitHub passwords are not stored in the system.
- **Clear access control:** Only authorized repositories are accessible.
- **Easy revocation:** Users can revoke access at any time on GitHub.
- **Modern compliance:** Compatible with GitHub's security policies.

This mechanism ensures LivingDocs maintains strong GitHub integration while keeping user data and accounts secure.

2.7.3 Repository Management

After successfully connecting to GitHub, LivingDocs allows users to select repositories to manage and synchronize with the documentation system.

Repository Selection The system uses the GitHub REST API to retrieve the list of repositories the user can access. Users can select one or multiple repositories to include in the LivingDocs workspace.

Stored information includes:

- Repository ID.
- Repository name.
- Owner.
- Default branch.
- Last synchronization time.

Data Synchronization from Main Branch Once connected, users can choose repositories to manage. Repository information is stored for tracking and synchronization:

- Update the latest commit.
- Retrieve the list of changed files.
- Synchronize merged Pull Requests.
- Update repository metadata.

Update Status Tracking The system maintains synchronization status through indicators such as:

- Last processed commit.

- Open Pull Requests.
- Merged Pull Requests.
- Last synchronization time.
- Success or failure status of synchronization.

Currently, this content is at the design and solution description stage. Implementation of repository connection and synchronization will be carried out once the Backend and Database components are completed.

2.7.4 CI/CD Integration with GitHub Actions

To support development in later stages, LivingDocs plans to integrate with **GitHub Actions** to automate the **Continuous Integration / Continuous Deployment (CI/CD)** process. This integration will be implemented once Backend, Frontend, and Database components are ready.

Automated Build/Test Workflow During system deployment, when a new commit is pushed or a Pull Request is created, GitHub Actions can be configured to trigger automated workflows. The process typically includes:

1. Checkout source code.
2. Set up runtime environment.
3. Build the project.
4. Run unit tests and integration tests.
5. Send results back to GitHub.

Automatic Documentation Update on PR Merge After a Pull Request is merged into the main branch, the workflow can call LivingDocs API to:

- Analyze source code changes.
- Identify related documentation to update.
- Generate documentation update proposals.
- Synchronize new documentation versions into the system.

Benefits of CI/CD

- **Reduce manual errors:** Avoid forgetting documentation updates or missing checks.
- **Accelerate development:** Automated checks detect errors earlier.
- **Ensure quality:** All changes are built and tested before merging.
- **Maintain synchronization:** Documentation is always updated with source code changes.

In later stages, the team plans to integrate GitHub Actions to support CI/CD workflows.

2.7.5 Pull Request and Commit Tracking

Tracking Pull Requests and commits is essential for LivingDocs to link source code changes with corresponding technical documentation.

Recording Changes from Pull Requests When GitHub sends a `pull_request` event, the system will:

1. Receive the webhook from GitHub.
2. Verify the webhook's security signature.
3. Retrieve the list of added, modified, or deleted files.
4. Classify the impact level on documentation.
5. Create a tracking record in LivingDocs.

Linking Commits to Documentation Each commit is associated with:

- Affected system modules or components.
- Changed APIs or classes.
- Related design or API documentation.
- Documentation version requiring update.

This mechanism enables traceability between documentation and commits, supporting auditing and system maintenance.

Review and Merge Workflow

1. Developer creates a Pull Request.
2. GitHub Actions automatically build and run tests.
3. LivingDocs analyzes documentation impact.
4. Reviewer checks source code and proposes documentation updates.
5. Once approved, the Pull Request is merged into the main branch.
6. The system triggers final documentation synchronization.

This workflow ensures all important source code changes are reviewed alongside corresponding documentation before integration.

2.7.6 Security and Error Handling

Since GitHub Integration works with external services and critical source code, the system requires proper security and error handling mechanisms to ensure stability and safety.

Expired Tokens GitHub access tokens may expire or be revoked. When detected, LivingDocs will:

- Mark connection status as **EXPIRED**.
- Pause repository synchronization.
- Display a message requesting the user to reconnect GitHub.

GitHub Connection Errors Common errors include:

- Network disconnection.
- GitHub API not responding.
- Exceeding request limits (rate limit).
- Webhook delivery failure.

If GitHub API does not respond, the system retries multiple times before notifying the user.

Warning and Token Refresh Mechanism LivingDocs supports:

- Alerts when tokens are about to expire.
- Notifications to users via the system interface.
- If tokens are invalid or revoked, the system requests reconnection to GitHub.
- Logging all authentication and synchronization processes for troubleshooting.

Data Security During Synchronization To protect source code and credentials, the system applies:

- Tokens stored in encrypted form to prevent leaks.
- HTTPS/TLS for all GitHub connections.
- Verification of webhook **X-Hub-Signature-256** to prevent forgery.
- Role-Based Access Control (RBAC).
- No logging of tokens or sensitive data in application logs.

This chapter provides the foundation for implementing GitHub Integration in upcoming sprints. At the current stage, the team focuses on requirements, system design, and technical documentation before starting source code development.

2.8 AI-Driven Documentation Generation Engine

2.8.1 Introduction

Alongside source code management, Artificial Intelligence (AI) plays a core role in the **LivingDocs** system to automate the initialization, updating, and maintenance of technical documentation. By applying Large Language Models (LLMs), LivingDocs transforms

complex source code into understandable, accurate, and real-time synchronized documentation.

In the Software Engineering project, integrating the AI Engine into **LivingDocs** aims at the following goals:

- Automatically generate technical documentation from source code without continuous manual intervention from developers.
- Maintain contextual synchronization between actual source code and system documentation.
- Support the generation of architecture diagrams, API documentation, and user guides with high accuracy.
- Optimize documentation writing time, allowing the development team to focus on coding.
- Detect and alert on documentation sections that are inconsistent or outdated compared to newly updated source code.

Thanks to the combination of AI and repository monitoring, LivingDocs transforms documentation from a static file into a **living documentation system**, capable of evolving alongside the software product.

2.8.2 AI Architecture and Model Integration

To achieve high performance and optimal accuracy in processing programming languages, LivingDocs builds a flexible AI integration architecture, directly connected to advanced LLM services.

AI Models and Integrated Services The system connects to AI models via RESTful API/gRPC, leveraging leading models such as OpenAI GPT-4, Google Gemini Pro, or Anthropic Claude. These models are chosen for their ability to understand multi-language source code and generate high-quality natural text.

Context Window and Token Management Project source code often exceeds the context window limits of AI models. LivingDocs addresses this by:

- Splitting source code into chunks (**code chunking**) based on grammar structure (Module, Class, Function).
- Filtering information to only send relevant or changed code segments.
- Optimizing token usage to save costs and reduce response time.

Benefits of Flexible AI Architecture

- **Scalability:** Easily switch or integrate new AI models in the future.

- **High performance:** Handle multiple documentation requests in parallel across different modules.
- **Cost optimization:** Apply caching and smart token management.
- **Customization freedom:** Allow parameter tuning (`temperature`, `top_p`) depending on document type.

2.8.3 Automated Documentation Generation Process

The automated documentation generation process in LivingDocs combines static code parsing with the synthesis capabilities of the AI Engine.

Source Code Analysis and Structure Extraction Before sending data to AI, the system performs static code analysis to build an Abstract Syntax Tree (AST). This extracts core components such as:

- Functions, methods, parameters, and return types.
- Classes, interfaces, and their relationships.
- API endpoints, HTTP methods, and request/response structures.
- Inline comments available in the source code.

Automated Documentation Workflow

1. The system receives source code change events (via GitHub Webhook or user command).
2. The AST parser extracts structure and details.
3. The Context Builder aggregates source code with related existing documentation.
4. The AI Engine processes the request and generates documentation in the required format.
5. The Post-processor verifies Markdown/LaTeX formatting and syntax.
6. The new documentation is displayed in the UI and ready for Pull Request updates.

Types of Documentation Supported

- **API Specifications:** Detailed request, response, status codes, and examples.
- **Architecture Diagrams:** Generate Mermaid/PlantUML for class and sequence diagrams.
- **User Guides:** Step-by-step feature instructions for end users.
- **Changelog:** Summarize technical changes into understandable language for project managers.

2.8.4 Retrieval-Augmented Generation (RAG) System

A major challenge in applying LLMs to software development is hallucination or lack of global project context. LivingDocs solves this by implementing **Retrieval-Augmented Generation (RAG)**.

Concept of RAG in LivingDocs RAG combines semantic search with LLM text generation. Instead of sending isolated code, the system queries related documents and code across the project to provide additional context.

Vectorization and Embedding Workflow

1. All documents and source code are converted into vector embeddings via an **Embedding Model**.
2. Vector data is stored and indexed in a **Vector Database**.
3. When documentation is requested, the system queries the most semantically similar vectors.
4. Results are packaged as context and sent to the LLM.

Benefits of RAG

- **Superior accuracy:** Minimize hallucinations.
- **Global context understanding:** Documentation reflects module relationships.
- **Flexible knowledge updates:** No need to fine-tune AI models; just update the vector database.

2.8.5 Prompt Engineering and Quality Control

The quality of generated documentation depends heavily on prompt design and output quality control.

Structured Prompt Design LivingDocs builds a standardized prompt library based on strict design principles:

- **Role assignment:** AI acts as a System Architect or Technical Writer.
- **Format constraints:** Require AI to return Markdown or LaTeX/Mermaid.
- **Few-shot prompting:** Provide sample project documents for style and structure consistency.

Review and Quality Control Workflow

1. **Automatic syntax check:** Verify Markdown, LaTeX, Mermaid syntax.

2. **Source code alignment:** Compare parameters and function names with actual code.
3. **Human-in-the-loop approval:** Developers/Tech Leads review and approve before merging.

Continuous Improvement The system records developer edits to AI-generated content. Feedback is used to refine prompts, improving future documentation quality.

2.8.6 Security, Privacy and Error Handling

Using AI with enterprise source code requires strict standards for security, intellectual property protection, and error handling.

Source Code Security and Privacy

- **Zero Data Retention:** Connect via enterprise APIs where providers guarantee no training on user data.
- **Data masking:** Automatically detect and remove API keys, passwords, IP addresses, and PII before sending to AI.
- **Encrypted transmission:** Use HTTPS/TLS for all data exchanges.

API Error and Rate Limiting Handling

- **Rate limit handling:** Queue requests and retry with exponential backoff.
- **Timeout handling:** Set reasonable timeouts and cancel tasks to avoid system blockage.

Fallback and Offline Mode

- **Model fallback:** Switch to backup AI models (e.g., GPT-4 → Claude or local LLM).
- **Template-based documentation:** If AI services are unavailable, generate basic docs from AST.

Token Budgeting and Access Control

- Set token quotas per user/project to control costs.
- Apply Role-Based Access Control (RBAC) for AI documentation features.
- Log AI API usage history (without source code) for auditing and cost tracking.

These mechanisms allow **LivingDocs** to maximize the power of AI in automating documentation while maintaining reliability, security, and absolute safety for the system and user data.

Chapter 3

Documentation Drift Detection

3.1 Overview

The **Documentation Drift Detection** module within the LivingDocs system is designed to automatically identify inconsistencies (drift) between source code hosted on GitHub and its associated documentation. Whenever code changes occur via commits or Pull Requests, the system parses the code, performs semantic evaluation, and compares it against existing documentation to issue timely alerts.

This module aligns with the functional specifications defined in the system requirement documentation.

3.2 Objectives & Scope

- Intercept and process commit and Pull Request events via GitHub Webhooks.
- Analyze source code structure using Abstract Syntax Tree (AST) parsing and semantic analysis.
- Map and compare code entities with existing documentation artifacts.
- Calculate drift confidence scores and classify severity levels (Low, Medium, High, Critical).
- Automatically generate drift reports with grounding evidence and dispatch alerts to Staff and Technical Leads.

3.3 Functional Details

3.3.1 GitHub Event Ingestion (FR01)

The system uses Webhooks to receive real-time updates whenever a new commit is pushed or a Pull Request is opened or updated in a monitored repository.

3.3.2 AST Source Code Parsing (FR02)

Modified source code is processed through an AST parser to extract key technical entities:

- Class names, method signatures, interfaces, and package hierarchies.
- Input parameter lists, data types, return values, and exceptions.
- Associated docstrings, inline comments, and annotations.

3.3.3 Code-Doc Comparison & Scoring (FR03, FR04)

The system retrieves documentation linked to the modified source files and evaluates consistency across three drift categories:

- **Referential Drift:** Referenced classes, methods, or parameters have been deleted, moved, or renamed.
- **Signature Drift:** Method signatures, parameters, or return types have changed while the name remains identical.
- **Semantic Drift:** Internal business logic or component behavior has changed while signatures remain unchanged.

Drift severity is classified into four levels:

- **Low (No/Minor Drift):** Cosmetic changes, inline comment fixes, or internal variable renames.
- **Medium (Moderate Drift):** Internal logic updates affecting module behavior without breaking external contracts.
- **High (Signature Drift):** Altered function parameters or return signatures not yet updated in docs.
- **Critical (Breaking Drift):** Deleted/renamed core APIs, major architectural refactoring, or critical config changes.

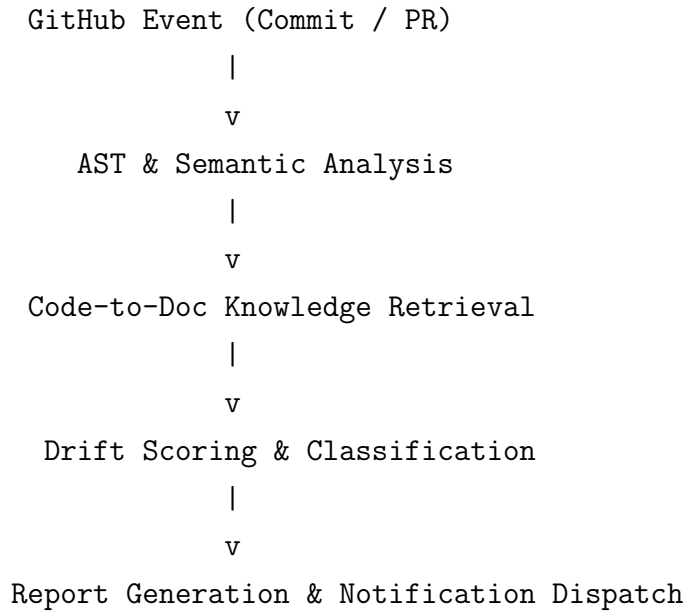
3.3.4 Report Generation & Alerting (FR05, FR06)

The system automatically generates a detailed drift report comprising:

- Commit hash or Pull Request ID.
- Affected source files and corresponding documentation links.
- Grounding evidence (Code Diff, Commit context, and Jira ticket metadata).
- Drift severity level, AI confidence score, and suggested update draft.

Alerts are delivered to Staff reviewers or Technical Leads via the system web interface, Slack, or email notifications.

3.4 Processing Workflow



3.5 Acceptance Criteria

1. Accurately receive and process Webhook event payloads from GitHub.
2. Successfully parse modified source code and extract code entities via AST.
3. Correctly detect and classify drift severity (Referential, Signature, Semantic).
4. Generate comprehensive drift reports accompanied by grounding evidence and alert designated reviewers.

Chapter 4

Automatic Documentation Update on Source-Code Change

Documentation drift occurs frequently in software engineering when code evolves continuously through Commits and Pull Requests while technical documentation remains untouched. The **Automatic Documentation Update** module in LivingDocs addresses this issue by automatically maintaining synchronization between source code changes and project documentation.

4.1 Module Overview & Objectives

The primary objective of this module is to automate the end-to-end process of detecting code modifications and updating affected documentation artifacts:

- **Real-Time Event Detection:** Intercept source code modification events dispatched via GitHub Webhooks (Push Commits, PR creations/updates, and PR merges).
- **AST & AI-Driven Generation:** Utilize Abstract Syntax Tree (AST) parsing alongside LLM and RAG capabilities to identify affected code entities (Classes, Methods, Endpoints, Parameters) and draft updated documentation versions.
- **Governed Workflow Integration:** Route automatically updated documentation drafts into the multi-tiered approval pipeline (AI → Staff Reviewer → Manager Approver) prior to publishing.
- **Side-by-Side Diff & Rollback:** Provide interactive side-by-side diff views, track version change logs, and enable single-click version rollbacks.

Table 4.1: Functional Requirements Matrix for Automatic Documentation Update

Requirement ID	Feature Name	Functional Description	Priority
FR-AU-01	Auto-Update Trigger	Automatically detect code changes via Commit/PR, identify affected docs, and generate draft versions.	Critical
FR-AU-02	Auto-Update Toggles	Enable or disable auto-update features per Repository, Module, or specific Document type.	High
FR-AU-03	Auto-Updated View	Display a list of documentation drafts automatically updated by the AI engine following code changes.	High
FR-AU-04	Grounding Evidence Linkage	Link each auto-updated document draft to its triggering Commit Hash, Author, and Code Diff.	High
FR-AU-05	Side-by-Side Diff View	Provide a visual line-by-line diff comparing the AI-generated draft against the currently published version.	Critical
FR-AU-06	Developer Actions	Allow developers to Accept (forward to review), Edit directly, or Dismiss AI-suggested updates.	Critical
FR-AU-07	On-Demand Re-update	Allow developers to manually trigger re-generation for a specific code entity’s documentation when required.	Medium
FR-AU-08	Document Rollback	Enable authorized users to roll back a document to the state prior to an auto-update execution.	Critical

4.2 Functional Requirements

4.3 Detailed Use Cases

4.3.1 UC-AU-01: Automatic Update Triggering on Source Code Change

- **Actors:** System (GitHub Webhook / Event Bus), Developer.
- **Pre-conditions:** The repository is connected to LivingDocs and auto-update policies are active.
- **Main Flow:**
 1. GitHub sends a webhook payload (`push` or `pull_request`) upon code changes.
 2. The `CodeAnalysisService` parses AST diffs to identify modified code entities.
 3. The `RagRetrievalService` queries the Knowledge Base to locate linked documentation chunks.

4. The AI Engine processes the Code Diff, Document Context, and Template rules to generate an updated draft.
5. The system stores the draft under `PENDING_DEV_ACCEPTANCE` status and notifies the developer.

4.3.2 UC-AU-02: Diff Comparison & Developer Acceptance

- **Actors:** Developer / Author.
- **Pre-conditions:** An AI auto-updated draft is pending review.
- **Main Flow:**
 1. The developer opens the side-by-side diff view.
 2. The interface highlights added, modified, or removed sections.
 3. The developer executes one of the following actions:
 - **Accept:** Approves the draft and forwards it to the Staff Review Queue.
 - **Edit:** Modifies the Markdown draft directly before submitting.
 - **Dismiss:** Rejects the AI proposal if no documentation update is necessary.

4.4 System Architecture & Processing Flow

The auto-update module relies on an event-driven microservices architecture:

[GitHub Webhook] ---> (POST /api/v1/webhooks/github) ---> [Spring Boot Backend]

1. Validate HMAC signature & extract Commit SHA / Code Diff.
2. Publish message to Kafka Topic: "code-change-events".

[Kafka Consumer] ---> [Python AI Engine (FastAPI)]

1. Parse AST to detect function signature/behavioral changes.
2. Execute RAG to pull relevant documentation chunks.
3. Prompt LLM to update documentation maintaining template structure.

[Spring Boot Backend]

1. Persist new record in 'document_versions' with status 'PENDING_DEV'.
2. Create audit trail record in 'document_change_logs'.
3. Dispatch WebSocket notification to developer interface.

4.5 Database Schema

The module maintains history, change logs, and configuration state using PostgreSQL tables:

Listing 4.1: PostgreSQL Database Schema for Auto Update Module

— *Document Versions Entity*

```
CREATE TABLE document_versions (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    document_id UUID NOT NULL REFERENCES documents(id) ON DELETE CASCADE,  
    version_number VARCHAR(20) NOT NULL,  
    content TEXT NOT NULL,  
    created_by_role VARCHAR(20) NOT NULL, — 'AI', 'DEVELOPER', 'STAFF',  
    trigger_type VARCHAR(30) NOT NULL, — 'COMMIT', 'PULL-REQUEST', 'P  
    trigger_commit_hash VARCHAR(40),  
    confidence_score FLOAT,  
    status VARCHAR(30) NOT NULL, — 'PENDING_DEV', 'IN_REVIEW',  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

— *Audit & Change Log Entity*

```
CREATE TABLE document_change_logs (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    document_id UUID NOT NULL REFERENCES documents(id) ON DELETE CASCADE,  
    from_version_id UUID REFERENCES document_versions(id),  
    to_version_id UUID REFERENCES document_versions(id),  
    acting_role VARCHAR(20) NOT NULL, — 'AI', 'STAFF', 'MANAGER'  
    action_type VARCHAR(50) NOT NULL, — 'AUTO.UPDATE.GENERATED', 'RO  
    change_summary TEXT,  
    grounding_evidence JSONB,  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

Chapter 5

Knowledge Base & Vector Retrieval Management

The **Knowledge Base** serves as the central semantic memory for the LivingDocs system. It is responsible for indexing, vectorizing, and retrieving documentation chunks to support Retrieval-Augmented Generation (RAG) during automated documentation updates and drift detection workflows.

5.1 Module Overview & Objectives

The Knowledge Base module manages both raw documentation artifacts and their high-dimensional vector embeddings to enable fast, accurate semantic searches. Key functional objectives include:

- **Automated Chunking & Ingestion:** Parse incoming Markdown/LaTeX documentation into semantic chunks and generate vector embeddings via embedding models.
- **Vector Store Management:** Index chunk embeddings into a high-performance vector database (e.g., Qdrant / Pgvector) for efficient similarity searches.
- **Semantic RAG Retrieval:** Retrieve relevant context based on code diff keywords, Abstract Syntax Tree (AST) entities, and natural language queries.
- **Audit Trail & History Tracking:** Maintain full version lineage for each knowledge chunk to support change auditing and seamless version rollbacks.

Table 5.1: Functional Requirements for Knowledge Base Module

Requirement ID	Feature Name	Functional Description	Priority
FR-KB-01	Document Indexing	Automatically chunk and vectorize updated or newly published documents into the Vector Store.	Critical
FR-KB-02	Hybrid Semantic Search	Execute combined dense (vector) and sparse (keyword) queries to fetch grounding context for LLM generation.	Critical
FR-KB-03	Linkage Mapping	Maintain bidirectional mappings between code AST symbols (classes, functions) and document chunks.	High
FR-KB-04	Chunk Versioning	Track chunk-level history, maintaining active/deprecated states for out-of-date knowledge.	High
FR-KB-05	Knowledge Maintenance	Provide APIs for manual or automated re-indexing and pruning of stale vector embeddings.	Medium

5.2 Functional Requirements Matrix

5.3 Knowledge Processing & Retrieval Pipeline

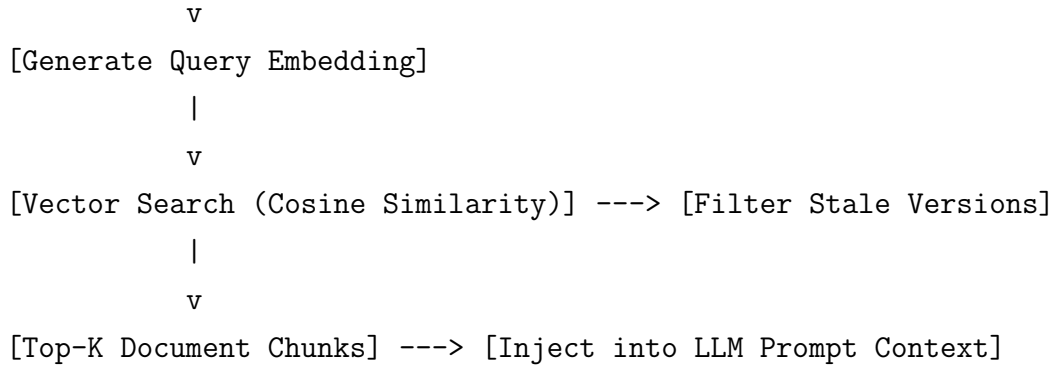
5.3.1 Document Ingestion Flow

When a document is modified or auto-updated, the Knowledge Base ingestion pipeline executes the following steps:

1. **Preprocessing & Parsing:** Parse document syntax (Markdown / LaTeX) and break text into optimal semantic chunks (250–500 tokens with overlapping boundaries).
2. **Embedding Generation:** Pass extracted chunks to the embedding model to produce 1536-dimensional vector representations.
3. **Metadata Attachment:** Associate each chunk vector with metadata including `document_id`, `version_id`, `file_path`, and linked `ast_entities`.
4. **Vector Upsert:** Store vectors and payload metadata in the Vector Store.

5.3.2 RAG Retrieval Sequence

```
[AST / Code Diff Trigger]
|
v
[Extract Key Entities & Context]
|
```



5.4 Database Schema

The Knowledge Base relies on PostgreSQL (with `pgvector` extension) or dedicated vector databases to manage chunk metadata and version relations.

Listing 5.1: PostgreSQL Database Schema for Knowledge Base Chunks

— *Knowledge Base Document Chunks*

```

CREATE TABLE document_chunks (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    document_id UUID NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
    version_id UUID NOT NULL REFERENCES document_versions(id) ON DELETE CASCADE,
    chunk_index INT NOT NULL,
    content TEXT NOT NULL,
    embedding vector(1536), — Vector embedding representation
    linked_symbols JSONB, — Stores AST entities: ["com.livingdocs.server"]
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

— *Index for Fast Vector Similarity Search (HNSW)*

```

CREATE INDEX idx_document_chunks_embedding
ON document_chunks
USING hnsw (embedding vector_cosine_ops);

```

Chapter 6

System Administration (Administrator)

The System Administration module plays a pivotal role in maintaining system operations, ensuring security and compliance, managing role-based access control, and configuring Artificial Intelligence (AI Engine) parameters for the entire **LivingDocs** platform.

6.1 User Management & Role-Based Access Control (RBAC)

The system enforces a strict Role-Based Access Control (RBAC) model to regulate access permissions for every organization member:

- **Account & Role Governance:**
 - Create, update, activate, or suspend user accounts within the system.
 - Assign and adjust user roles across 5 permission levels: *Developer (Author)*, *Staff (Reviewer)*, *Manager (Approver)*, *Technical Lead (Documentation Owner)*, and *Administrator*.
- **Workspace & Repository Management:**
 - Define and allocate Workspaces for engineering teams.
 - Grant repository access permissions to specific user groups or teams.

6.2 Platform & Version Control System Integration

Administrators are responsible for establishing secure connections with the Version Control System (VCS) infrastructure and third-party developer tools:

- **GitHub App & OAuth Connection Configuration:**

- Configure Client ID, Client Secret, and Private Keys for GitHub App and OAuth2/OIDC connections.
- Manage repository access scopes requested during the GitHub App installation.
- **GitHub Actions & Webhook Integration:**
 - Set up Webhook URLs to receive real-time event payloads from GitHub (Commits, Pull Requests, Merge events).
 - Manage Webhook Secret Keys to verify signature payload authenticity.
- **External Tool Integrations:**
 - Configure connections with task management tools (e.g., Jira) to extract issue context associated with code changes.
 - Set up notification channels (Slack, Microsoft Teams) for documentation drift alerts and pending approval reviews.

6.3 AI Engine Service Configuration

Administrators can fine-tune AI Engine parameters to balance documentation output accuracy and operational API costs:

- **Language Models & Embeddings:**
 - Select Large Language Models (LLMs) used for code analysis and documentation generation (e.g., GPT-4o, Claude 3.5 Sonnet, or open-source models such as Llama 3).
 - Configure Vector Embedding models used in Retrieval-Augmented Generation (RAG).
- **AI Operational Parameters:**
 - *Confidence Threshold:* Set minimum confidence scores required before AI can suggest documentation updates automatically.
 - *Token Budget:* Define token usage limits per task or workspace to manage LLM API expenditures.
 - *Supported Languages:* Manage the list of programming languages supported for Abstract Syntax Tree (AST) parsing (e.g., Java, Python, TypeScript, Go, C++).

6.4 Auto-Update Policies, Audit Logs & System Operations

- **Auto-Update Governance:** Configure automatic documentation update triggers (on Commit, on PR, or on Merge) and set approval enforcement rules (route to Staff/Manager review vs. auto-apply after approval).

- **Change Log & Audit Trail Retention:** Set retention policies for document version histories and system audit logs to support compliance and traceability.
- **System Monitoring & Infrastructure Health:**
 - Monitor runtime performance across Microservices (Spring Boot), AI Service (FastAPI), Message Broker (Kafka), and Database (PostgreSQL).
 - Manage event queues and inspect Dead Letter Queues (DLQ) for failed background AI tasks recovery.

Chapter 7

System Testing & Quality Assurance Plan

7.1 Introduction

Software testing is a critical phase in the software development lifecycle, ensuring that the system operates strictly according to functional requirements, meets established quality benchmarks, and minimizes potential operational defects prior to production release.

For the **LivingDocs** platform, quality assurance plays an indispensable role due to the system's architecture, which integrates multiple loosely coupled modules: GitHub Webhook Integration, AST Parsing, Retrieval-Augmented Generation (RAG), Vector Databases, and Multi-Tier Governance Workflows. An unhandled exception or data mismatch in any upstream module can compromise automated documentation synchronization.

This chapter outlines the comprehensive testing strategy, environmental configurations, test scenarios, and concrete execution results designed to validate the LivingDocs platform.

7.2 Testing Objectives & Scope

7.2.1 Testing Objectives

The quality assurance process for LivingDocs is structured around the following goals:

- **Functional Verification:** Ensure all core features operate as defined in the functional requirements matrix.
- **Integration Integrity:** Validate seamless data transfer across GitHub Webhooks, Spring Boot Core Services, Python AI Engines, and Vector Databases.
- **Accuracy of AI Generation:** Evaluate the precision and context alignment of

documentation updates generated via RAG and LLM prompts.

- **Drift Detection Accuracy:** Verify the system's ability to accurately classify drift severity levels (Low, Medium, High, Critical) based on AST code changes.
- **Fault Tolerance & Stability:** Test system behavior under error conditions, API rate limits, and unparseable source code diffs.

7.2.2 Testing Scope

The testing scope encompasses all functional modules of LivingDocs:

- **GitHub Integration:** OAuth2 authentication, repository synchronization, and Webhook event delivery (Push, PR).
- **AST & Code Analysis:** Abstract Syntax Tree parsing for detecting modified classes, method signatures, parameters, and return types.
- **Documentation Drift Detection:** Identification and severity scoring of Referential, Signature, and Semantic drifts.
- **AI Generation & RAG Pipeline:** Vector search retrieval, prompt assembly, and Markdown documentation generation.
- **Automatic Documentation Update:** Automated draft generation, side-by-side diff comparison, and developer acceptance actions.
- **Review & Approval Workflow:** Multi-tier governance routing (AI → Staff Reviewer → Manager Approver), comment logging, and publishing to GitHub.
- **Knowledge Base & Versioning:** History tracking, change log audits, and single-click document rollback mechanisms.

7.3 Testing Strategy & Levels

LivingDocs employs a multi-layered testing strategy to detect defects early and ensure system reliability:

- **Unit Testing:** Conducted at the code level using JUnit 5 and Mockito for Java backend services, and pytest for Python AI modules.
- **Integration Testing:** Validates API interactions, event consumption via Kafka, and database persistence between Spring Boot and FastAPI microservices.
- **System Testing:** End-to-end evaluation of complete workflows, simulating real GitHub Webhook payloads to final document publishing.
- **Acceptance Testing (UAT):** Execution of real-world scenarios to confirm the platform meets functional expectations for Developers, Staff Reviewers, and Managers.

7.4 Testing Environment & Tools

Testing is conducted in an isolated environment mirroring the production setup to prevent data corruption and ensure reproducible results.

Table 7.1: Testing Environment Specifications

Component	Testing Technology / Configuration
Backend Services	Java 17, Spring Boot 3.x, Python 3.11 (FastAPI)
Frontend Interface	ReactJS (TypeScript), TailwindCSS
Database Systems	PostgreSQL 15 (Metadata & Versioning), Qdrant / Pgvector
Event Broker	Apache Kafka (Mocked/Local Container)
External API Mocks	Mock GitHub Webhook Payload Generator, OpenAI API Mocks
Testing Frameworks	JUnit 5, Mockito, PyTest, Postman, Jest

7.5 Defect Management Workflow

Discovered bugs and discrepancies are processed through a structured lifecycle:

- Logging:** Defects are logged with reproduction steps, error logs, and severity classifications.
- Triage & Assignment:** Issues are classified into severity levels (*Critical, High, Medium, Low*) and assigned to the responsible module developer.
- Remediation & Retesting:** Fixed defects are deployed to the test environment and re-tested by QA.
- Closure:** Once verified, the defect is officially marked as Closed.

7.6 Detailed Test Cases

The following test cases demonstrate the execution and verification of key features within the **Documentation Drift Detection** and **Review Queue & Approval Workflow** modules.

TC-RQ-001: Filter Review Queue Dashboard by Drift Severity

Component	Test Case Specification
Test ID	TC-RQ-001
Module	Review Queue Dashboard
Objective	Verify that the system correctly filters pending documentation reviews by their drift severity level (<i>Critical, High, Medium, Low</i>).

Pre-conditions	Staff reviewer is authenticated. The Review Queue contains at least 5 pending document drafts across various severity levels.
Input Data	Select filter tag: CRITICAL .
Execution Steps	1. Navigate to the Review Queue Dashboard. 2. Locate the "Filter by Drift Severity" toolbar. 3. Click the CRITICAL severity filter button. 4. Observe the updated data grid.
Expected Result	The queue updates immediately to show only items marked with CRITICAL severity (highlighted with a red badge). All other items are filtered out.
Actual Result	Operating as expected (Verified on ReactJS prototype interface).
Status	PASS

TC-RQ-002: Staff Reviewer Evaluation and Approval

Component	Test Case Specification
Test ID	TC-RQ-002
Module	Side-by-Side Review Workspace
Objective	Verify that a Staff Reviewer can inspect side-by-side code/doc diffs, provide feedback, and approve a document from IN_REVIEW to APPROVED .
Pre-conditions	A document draft is in IN_REVIEW status. User is logged in with Staff Reviewer privileges.
Input Data	Feedback text: <i>"Documentation matches modified API signatures accurately."</i> ; Click [Approve] button.
Execution Steps	1. Click "Review" on a pending document entry in the Review Queue. 2. Inspect the Side-by-Side Workspace displaying Code Diff and Grounding Evidence. 3. Enter review feedback into the comment field. 4. Click the [Approve] button.

Expected Result	- Document status transitions from IN_REVIEW to APPROVED. - Toast notification appears: <i>"Approved and forwarded to Manager successfully."</i> - System redirects the user back to the active Review Queue.
Actual Result	Operating as expected (Verified on ReactJS frontend linked with API mocks).
Status	PASS

TC-RQ-003: Final Manager Approval and GitHub Publication

Component	Test Case Specification
Test ID	TC-RQ-003
Module	Manager Approval & GitHub Publishing Service
Objective	Verify that a Manager can grant final approval to publish an APPROVED document directly to the GitHub repository.
Pre-conditions	Document status is APPROVED. User is logged in with Manager role privileges.
Input Data	Click [Publish Document] button on the Manager approval dashboard.
Execution Steps	1. Access the Manager Approval queue. 2. Select a target document in APPROVED state. 3. Click [Publish Document].
Expected Result	- Document status updates to PUBLISHED. - The backend invokes the GitHub API service to commit the updated Markdown file to the designated repository path. - An Audit Trail log entry is created recording the Manager ID, timestamp, and commit hash.
Actual Result	Operating as expected (Verified with simulated GitHub Webhook/API integration).
Status	PASS

7.7 Summary

The testing framework established for LivingDocs guarantees that every component—from GitHub event ingestion to AI generation and multi-tier approval—is systematically verified. The successful execution of test cases demonstrates that the system achieves high reliability, accurate drift detection, and governed documentation maintenance.

7.8 Conclusion

7.8.1 Project Summary

In the context of modern software development, maintaining technical documentation that is always synchronized with source code is a major challenge for development teams. Many projects face the issue of documentation quickly becoming outdated due to infrequent updates, causing difficulties in maintenance, expansion, and system handover.

The **LivingDocs** project was proposed to address this problem by combining modern technologies such as GitHub Integration, Artificial Intelligence (AI), Retrieval-Augmented Generation (RAG), and Workflow Automation. The system aims to automate the creation, updating, and management of technical documentation based on source code changes, thereby significantly reducing manual workload and improving documentation quality.

Within the scope of the Software Engineering course, the team conducted requirement analysis, designed the overall architecture, and developed descriptive documentation for the main system components. These deliverables form the foundation for future development and deployment phases.

7.8.2 Project Achievements

During the project, the team accomplished the following key tasks:

- Analyzed functional and non-functional requirements of the system.
- Designed GitHub integration workflow using OAuth2 and GitHub API.
- Built an automated documentation update process triggered by GitHub events.
- Proposed an AI architecture to support source code analysis and technical documentation generation.
- Designed a version management mechanism for documentation and detection of inconsistencies between documentation and source code.
- Developed testing procedures and a system quality evaluation plan.

These results contribute to forming a modern documentation management model that effectively supports software development in a systematic way.

7.8.3 Project Limitations

Due to time constraints and the scope of the course, the project still has certain limitations:

- The system remains at the design and solution description stage, without full implementation of all functions.
- Some components such as the AI Engine, Vector Database, and Workflow Automation have not been tested in real environments.
- Performance, scalability, and security evaluations have not yet been conducted.
- Full integration with CI/CD processes in real development environments has not been implemented.

These limitations will serve as the basis for further improvement in subsequent development phases.

7.8.4 Future Development

In the future, the LivingDocs system can be expanded in various directions to enhance usability and practical application. Potential development directions include:

- Full implementation of Backend, Frontend, and Database components.
- Integration of AI models with more accurate source code analysis capabilities.
- Extending support to multiple source code management platforms such as GitLab or Bitbucket.
- Adding automated testing and continuous deployment (CI/CD) mechanisms.
- Developing a more detailed role-based access control system.
- Optimizing system performance for large-scale projects.
- Strengthening security and monitoring solutions during operation.

Advancing along these directions will help LivingDocs become an intelligent technical documentation management platform, better meeting the needs of real-world software development teams.

7.8.5 Conclusion

Overall, the **LivingDocs** project has proposed a solution to support technical documentation management in an automated, intelligent, and synchronized manner with the software development process. Through GitHub integration, AI application, and workflow automation, the system aims to reduce manual work, improve documentation accuracy, and enhance collaboration efficiency within development teams.

Although the project is currently at the design and documentation stage, the achieved results demonstrate the feasibility of the solution and lay the groundwork for future

implementation. With the proposed development directions, LivingDocs has the potential to become a modern documentation management tool, contributing to improved quality in software development and maintenance in real-world projects.

Bibliography